LangGraph Quiz

Section A: MCQs (1 mark each)

*1. **What is LangGraph primarily used for?***
a) Data visualization
b) Building graph-based AI workflows
c) Database management
d) Web scraping

Ans - B

---

*2. **In LangGraph, nodes represent:***
a) Database tables
b) Functions or steps in a workflow
c) UI components
d) API keys

Ans - B

---

*3. **What is the main advantage of graph-based workflows?***
a) Linear execution only
b) No need for state
c) Flexible branching and looping
d) Faster internet speed

Ans - C

---

*4. **What does "state" represent in LangGraph?***
a) Server location
b) Memory shared across nodes
c) Programming language
d) UI design

Ans - B

---

*5. **Which component decides the next step in a LangGraph workflow?***
a) Tokenizer
b) Edge
c) Prompt
d) Datase

Ans - B

---

*6. **A multi-step agent is best described as:***
a) A single API call
b) A system executing multiple connected steps
c) A database query
d) A static webpage

Ans - B

---

*7. **What is the purpose of conditional edges in LangGraph?***
a) To delete nodes
b) To control flow based on conditions
c) To store data
d) To train models

Ans - B

---

*8. **Which of the following is required to define a graph in LangGraph?***
a) Nodes and edges
b) Only APIs
c) Only datasets
d) Only UI components

Ans - A

---

*9. **StateGraph in LangGraph is used for:***
a) Styling UI
b) Managing workflow state and transitions

c) Creating databases
d) Sending emails

Ans - B

---

*10. **What is the output of a node in LangGraph?***
a) Only text
b) Updated state
c) Images only
d) HTML code

Ans - B

📌 *Section B: Short Answer (2–3 marks each)*

11. Explain the concept of graph-based workflows in LangGraph.

*Ans - Graph-based workflows in LangGraph represent an AI system as a set of connected **nodes** and **edges**. Each node performs a specific task, and edges define how control moves from one step to another. This allows branching, looping, and conditional routing.*

---

12. What is state management and why is it important in multi-step agents?

*Ans - Graph-based workflows in LangGraph represent an AI system as a set of connected **nodes** and **edges**. Each node performs a specific task, and edges define how control moves from one step to another. This allows branching, looping, and conditional routing.*

---

13. Differentiate between nodes and edges in LangGraph.

- ***Nodes:*** *Individual functions or processing steps in the workflow*
- ***Edges:*** *Connections that define the path from one node to another*

---

14. What is a StateGraph and how does it help in building agents?

*Ans - A **StateGraph** is a LangGraph structure that manages workflow state and transitions. It helps in building agents by storing shared memory and controlling how execution flows between steps.*

---

15. Give one real-world example where a multi-step agent can be useful.

*Ans - A **student assistant bot** that first takes a question, identifies whether it is about assignments or exams, routes it to the correct handler, and then generates a response.*

📌 *Section C: Coding-Based Questions (4–5 marks each)*

16. Consider the following code:

```
def step1(state):
    return {"msg": "Hello " + state["name"]}
```

👉 *What is happening in this function? Explain.*

*This function takes the current **state** as input. It accesses the name key from the state and returns an updated state dictionary containing a greeting message.*

Example:

state = {"name": "Shruti"}

Output:

{"msg": "Hello Shruti"}

---

17. Write a simple LangGraph workflow with two steps:

- *Step 1: Take user input*
- *Step 2: Generate a response*

(You can write pseudocode)

```
from langgraph.graph import StateGraph, END

def step1(state):
return {"question": input("Enter question: ")}
```

```
def step2(state):
return {"response": "Answer to: " + state["question"]}


graph = StateGraph(dict)
graph.add_node("input", step1)
graph.add_node("response", step2)
graph.set_entry_point("input")
graph.add_edge("input", "response")
graph.add_edge("response", END)
```

---

18. How would you add a conditional step in a LangGraph workflow? Explain with logic.

Conditional steps are added using ***conditional edges****.*

Example logic:

```
if state["type"] == "exam":
return "exam_handler"
else:
return "assignment_handler"
```

This helps route the workflow based on a condition.

📌 *Section D: Case Study (5–8 marks)*

19. Design a multi-step agent using LangGraph for a "Student Assistant Bot" that:

- *Takes a question*
- *Checks if it is about assignments or exams*
- *Routes to the correct handler*
- *Returns a response*

👉 *Explain:*

- *Nodes*
- *Edges*

- *State*

Ans - Student Assistant Bot Design

Nodes

- ***Input Node:*** *Takes student question*
- ***Classifier Node:*** *Checks whether question is about assignments or exams*
- ***Assignment Handler:*** *Handles assignment queries*
- ***Exam Handler:*** *Handles exam queries*
- ***Response Node:*** *Returns final response*

Edges

- *Input → Classifier*
- *Classifier → Assignment Handler / Exam Handler*
- *Handler → Response*

State

```
{
"question": "",
"category": "",
"response": ""
}
```

Workflow

```
Question Input
↓
Classifier
↙ ↘
Assignment Exam
↓ ↓
Response Node
↓
END
```

📌 *Section E: Advanced Conceptual (5 marks)*

20. Compare traditional linear AI pipelines with graph-based agent workflows. Why is LangGraph better for complex systems?

Traditional Linear Pipeline

- *Steps execute in a fixed sequence*
- *No branching*
- *Difficult to handle dynamic workflows*
- *Limited flexibility*

Graph-Based Workflow

- *Supports branching and loops*
- *Uses shared state*
- *Better for multi-step AI agents*
- *More scalable and flexible*

Why LangGraph is better

LangGraph is better for complex systems because it allows ***dynamic routing, conditional logic, memory management, and multi-step reasoning****, which traditional linear pipelines cannot efficiently support.*